

# ripwire

## The ripgrep of AI context.

A zero-runtime-dependency C++23 CLI that maps any codebase into a ranked, deterministic call graph for coding agents — and puts a tripwire on every claim it emits.

### **rip** — the speed half

Structural answers in tens of milliseconds, from a call graph it built itself.

### **wire** — the honesty half

Unprovable totals ship labelled as floors. A zero means none found — never none exists.

// the problem

# Your agent reads the whole library to answer one question

Blind grep, then whole-file reads. Most of what enters the context window is never used — it is paid for anyway, on every step of every task.

And bigger context is not better context: irrelevant text actively degrades the answer. The job is selection — a small, ranked, structural map beats a pile of open files.

Speed caveat travels with the number: ripwire answers from a warm, pre-built index, and round 4 measured its index cost too — 0.31 s, tabulated beside the competitors' rather than omitted.

**0.108 s**

median answer, warm index — round 4, all arms one machine one day, N = 60

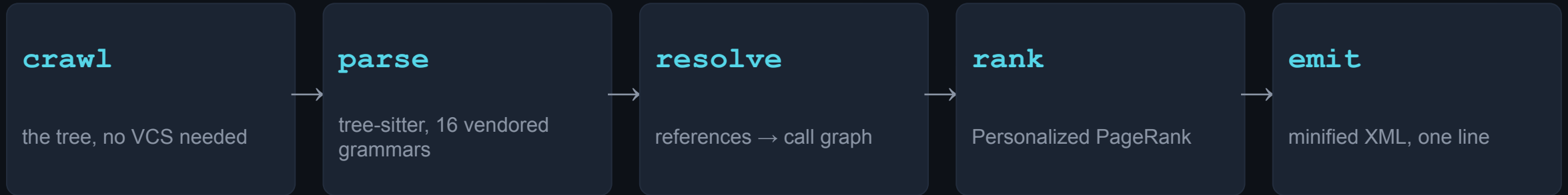
**−39.4%**

token ceiling vs the un-routed baseline — LocBench cost ledger, N = 243

// one binary, the whole pipeline

# Crawl → parse → resolve → rank → emit.

## Deterministic, end to end.



### byte-identical

two runs, same bytes — a gate on every push, not a tendency; warm equals cold

### zero runtime deps

CMake + a C++23 compiler; builds with the network off — vendored everything

### the languages

Rust · C++ · ObjC/C++ · C · Metal · CUDA · Python · Go · Swift · TypeScript · JavaScript · Java · Ruby · Bash · C# · JSON — Metal and ObjC ride a shared grammar

### agent-native

an MCP server and 156 long flags behind one `--help` that is always the authority

// every feature, one screen

# Seven families of questions it answers

|                 |                                                     |                                                                                                                                                          |
|-----------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| understand cold | <i>What is this repo, and what matters in it?</i>   | <code>--for --tree --lego --exemplar --recall --pack-task --token-budget</code>                                                                          |
| navigate        | <i>Who calls this? Safe to change? Which tests?</i> | <code>--callers --callees --uses --impact --path --connect --affected --situ --test-gate --from-trace</code>                                             |
| detail ladder   | <i>Show me more — but only where it pays.</i>       | <code>--detail --pack-signatures --outline --expand --compress</code>                                                                                    |
| quality & risk  | <i>Where is the risk, and did I just add some?</i>  | <code>--quality-panel --quality-delta --dmm --readability --ensemble --context-ratio --nonlocal-state --field-affinity --hotspots --lint --clones</code> |
| self-diagnosis  | <i>Is my setup actually working?</i>                | <code>--doctor --skipped</code>                                                                                                                          |
| security        | <i>Is this agent skill file safe to install?</i>    | <code>--scan-skill --scan-skills</code>                                                                                                                  |
| knobs & modes   | <i>Shape, format, cache, budget.</i>                | <code>--json --format --mcp</code>                                                                                                                       |

--help is generated from the binary's own flag table — 156 long flags; docs/COMMANDS.md documents 95 with recorded output

```
// reach for it at the moment, not the manual
```

# The reflexes: which verb fires when

## Landing cold on a task

```
--for="<task>" · --pack-task
```

ranked, budgeted context in one call

## Stack trace in hand

```
--from-trace
```

frames mapped to symbols, innermost body included

## “Is it safe to change X?”

```
--impact · --uses
```

the whole blast radius, not just 1-hop callers

## Just edited a symbol

```
--edit-check
```

did the contract change, who breaks — ~ms, warm

## About to write a helper

```
--exemplar · --grep
```

imitate the house pattern; catch the duplicate early

## Landing several branches

```
--merge-scout
```

pairwise conflict sites + a landing order

## Before you call it done

```
--quality-delta · --test-gate
```

only what you made worse; exactly which tests to run

## Handing the area on

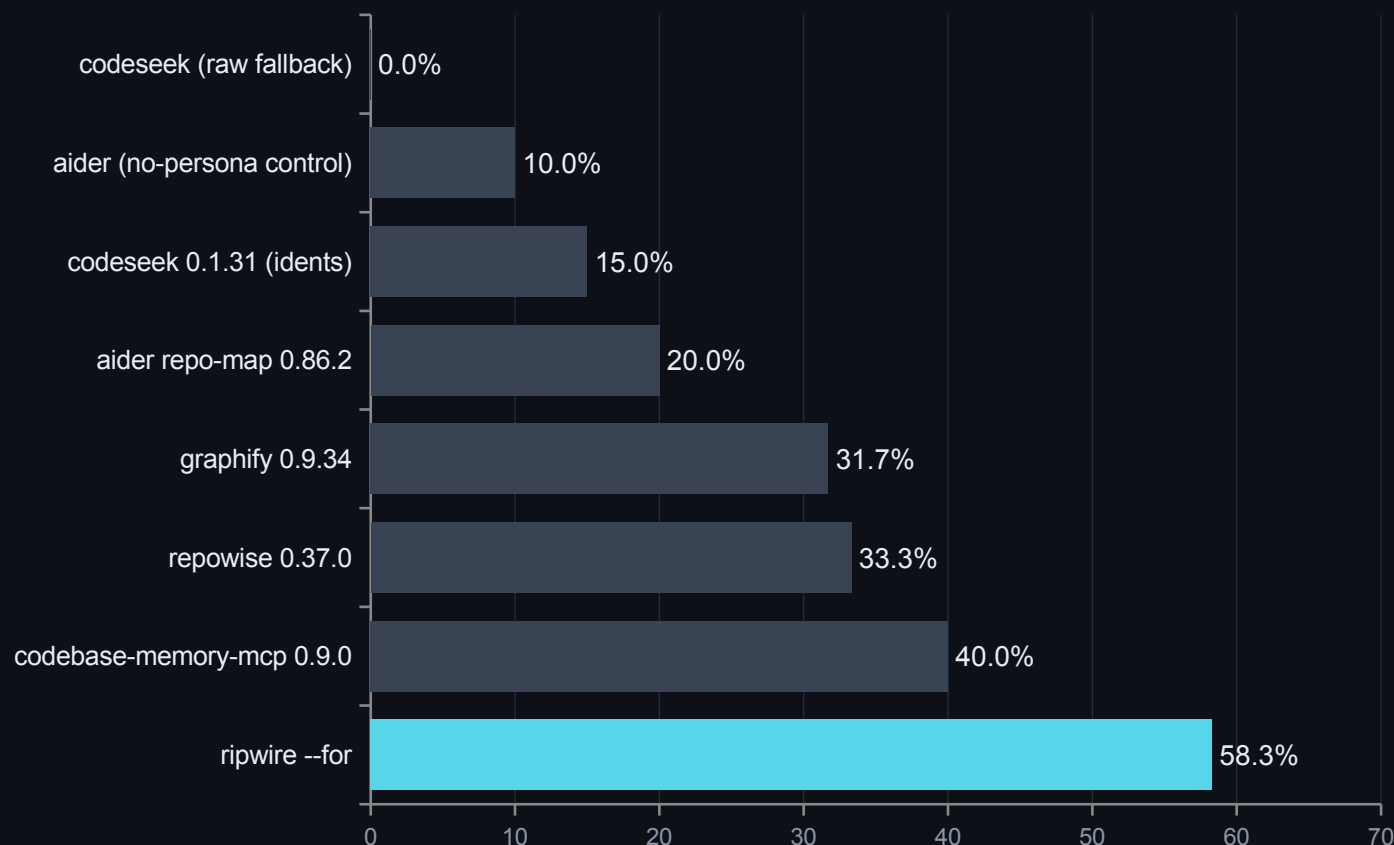
```
--handoff · --note-add
```

the verified-vs-heuristic packet the next agent needs

and `--pr-context` when you are reviewing someone else's diff — per-file blast radius, tests-to-run, hotspot flags

// one harness, one machine, one day — N = 60 paired, zero exclusions

# Head-to-head: every competitor, one table



1.46×

the margin over the best competitor — 58.3% against 40.0%  
strict file@10

## And the index it was measured with

ripwire 0.31 s · codebase-memory-mcp 1.24 s · codeseek  
3.37 s · graphify 7.82 s · repowise 34.0 s, whose worst  
single index was **352 s**. Cold, nothing to an answer: 0.213 s.

**What re-running cost us, published:** aider moved 18.3%  
→ 20.0% on its own tie-break nondeterminism, and we  
printed the higher number. Paired, ripwire loses 2 instances  
to codebase-memory-mcp, 2 to repowise, and 1 each to  
graphify, aider and the aider control.

bench/headtohead/r4-2026-08-06/ — harness, per-instance JSONL and recipe committed · replaces the two non-comparable tables r1 and r2  
needed

// scored against a third-party oracle, never against each other

# Graded by a compiler: zero silent misses

Both tools were scored against a scip-clang compiler-grade index of this repository — 68 answers, 34 blind-authored queries × --uses and --callers.

0

true silent misses — pre-fix and post-fix alike. Every imperfect answer either carried a discriminating self-flag (amb=, defs>1, external="1") or was an oracle-scope artifact where ripwire was right and the compiler could not see the file.

## What a compiler-grade index costs

|             | ripwire | Serena 1.6.2.dev0 (clangd 19.1.2) |      |
|-------------|---------|-----------------------------------|------|
| warm query  | 49.8 ms | 3,760 ms                          | ~70× |
| cold start  | 208 ms  | 8.7 s                             | ~40× |
| index build | none    | ~8 min                            | —    |
| errors      | 0 / 101 | 7 / 61                            | —    |

**Read honestly in both directions.** The LSP is more precise — site-level 0.929/0.941 against ripwire's 0.914/0.941 after the fix round, a gap of roughly 1.5 points with recall now equal. ripwire also pays ~7× the bytes per call, because its output carries self-describing legends. What it cannot do is answer a macro query at all — clangd's documentSymbol has no macro entries, which is 3 of its 7 errors — or start in less than eight minutes.

**The number that moved the wrong way, published with the rest:** over-hedging rose 18.6% → 22.6% across the fix round. Closing loss buckets added self-flags faster than it removed imperfect answers. That direction is deliberate — calibrated to warn too often rather than too rarely — but it is a cost, and it is on the slide.

// the rule: numbers are held back until the loss list is worked

# Every head-to-head round, and the ones that went against us

|     |               |                                                           |                                                    |
|-----|---------------|-----------------------------------------------------------|----------------------------------------------------|
| r1  | 2026-07-13/14 | Aider repo-map · codebase-memory-mcp · graphify           | superseded by r4                                   |
| r2  | 2026-08-03    | repowise · codeseek                                       | superseded by r4                                   |
| r3  | 2026-08-03    | headroom — a compression layer, so a different instrument | passed code through byte-identical                 |
| r4  | 2026-08-06/08 | all five competitors, one harness, one day                | the table on the previous slide                    |
| r6  | 2026-08-04/05 | agent-in-the-loop: the Codex CLI pilot                    | localization parity; +80% tokens, classified       |
| r7  | 2026-08-08    | CodeGraph, loss-first — we looked for our losses          | fix REJECTED by its own preregistered band         |
| r8  | 2026-08-08    | aiders again, at equal token budget                       | loss questions traced to one root cause            |
| r9  | 2026-08-09    | Serena / scip-clang oracle                                | 0 silent misses; the fix list it produced          |
| r10 | 2026-08-22    | GitNexus 1.6.9 — a graph-database context server          | 27-7-14 AFTER the fix list; first pass unpublished |

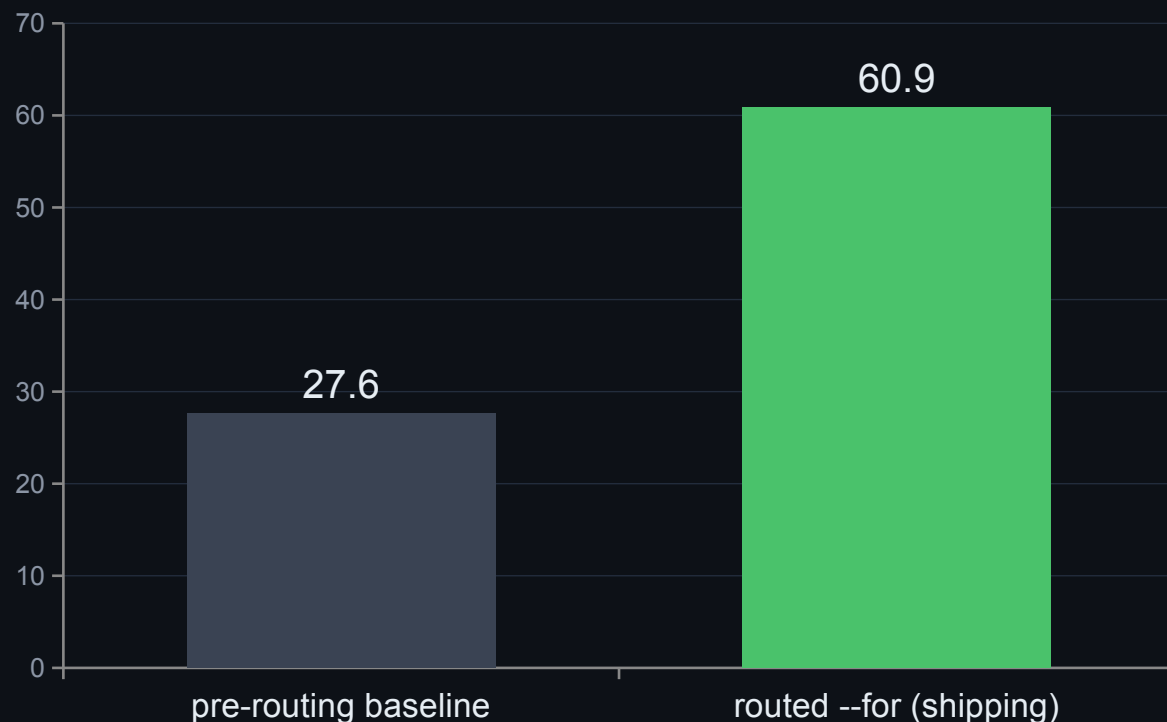
**r7 in full, because it is the point:** the fix was built, gated green, measured — predicted 12/22 against a pre-registered band of 10–14, measured 5/22 — and reverted rather than tuned. Published as a negative in docs/EVALS.md §7.

bench/headtohead/ (r1-r4, r9) · bench/r7/ · bench/r8/ — harnesses committed even for rounds that rejected their own fix · r5 left no head-to-head record in this tree, so it is not listed · r10's method and pins are published in docs/EVALS.md §2, its pre-fix numbers nowhere



// held-out, repo-disjoint, frozen dataset

# Routing the ranker: +33 points, measured properly



**+33.33pp**

paired delta, 243 held-out instances in 78 repository clusters

**+25.00pp**

clustered-bootstrap 95% lower bound — the honest floor

**-39.4%**

token ceiling for that gain

**+3.4%**

warm latency for that gain

// compiled, not interpreted — and measured on public corpora

# Fast enough to call on reflex

**0.213 s**

cold — nothing to an answer: parse, rank,  
reply, no cache (r4)

**0.108 s**

warm query, median — against aider's 2.920  
s (r4)

**0.31 s**

index build — against repowise's 34.0 s,  
worst case 352 s (r4)

**49.8 ms**

warm query on this repo — against a clangd  
LSP's 3,760 ms (r9)

## Where the second goes

Cold profiles are tree-sitter parse plus tag-query capture — not graph math. Resolve and PageRank together are a small fraction of the run, and the ranking was never the bottleneck.

Which is why the wins came from removing work, not micro-optimising it: demand-driven side captures, five whole-AST passes fused into one pre-order walk, a calibrated per-worker parse reserve.

## Two opt-in faster builds

LTO is on by default. PGO is a script away and buys 14–25% cold.

Both are opt-in performance, not opt-in correctness: **the output stays byte-identical across build flavours** — the determinism gate runs on both, and CI builds a plain flavour too, because NDEBUD once compiled a whole class of degrade-path checks out of existence.

// how it got there — the pre-release performance sprint

# Work removed before work optimized

## file ingest

FILE\* whole-file reads; CSV counting became a byte scan — iostream out of the hot path

## capture gating

value-use capture runs only for the verbs that actually need it, not on every run

## AST pass fusion

five whole-AST side-capture passes share ONE pre-order walk instead of five

## parallel parse

query compile overlaps parse scheduling; files schedule by size — ids stay deterministic

## radix + S+tree

numeric keys for edges and scores; a bounded-scratch hot map whose fixed cap reports its own overflow

## no std::map

unordered\_dense, reserve() before ingest — no std::unordered\_map on any hot path

**Why this slide carries no numbers.** The sprint's own headline pair was measured in June 2026 on a large private C++ corpus that is not publicly reproducible, and the deck does not quote figures a reader cannot re-derive — the previous slide's numbers all come from committed harnesses. What survives the omission is the **shape**: the graph math was never the bottleneck, and every win came from doing less work rather than doing the same work faster. Same map contract throughout — byte-identical output before and after each of these changes.

the mechanisms are in bench/PROFILE.md and the commit history; the private-corpus figures they produced are deliberately not quoted

// re-measured on this repository, 2026-08-08 — run any row yourself

# Ten everyday moments, and what each one costs

| Ask it                                 | Command                                | ripwire    | naive read | savings      |
|----------------------------------------|----------------------------------------|------------|------------|--------------|
| Orient me in this repo                 | <code>ripwire .</code>                 | ~5.6K      | ~20K–25K   | 3.6×–4.5×    |
| Where is X handled?                    | <code>--for="..."</code>               | ~1.9K      | ~4.9K–20K  | 2.6×–10.7×   |
| What do I already know?                | <code>--recall="..."</code>            | ~15K       | ~445K      | 29.2×        |
| Set me up for this task                | <code>--pack-task="..."</code>         | ~2.1K      | ~16K–80K   | 7.7×–37.7×   |
| Show me this one function              | <code>--expand=SYM --top-k=0</code>    | ~260–16.5K | ~43K–174K  | 2.6×–670×    |
| Who calls this function?               | <code>--callers=SYM</code>             | ~580       | ~40K–52K   | 69.2×–89.1×  |
| Is it safe to change this?             | <code>--impact=SYM + --uses=SYM</code> | ~1.3K      | ~18K       | 14.4×        |
| I have a stack trace                   | <code>--from-trace=FILE</code>         | ~1.4K      | ~124K–298K | 86.9×–208.6× |
| I changed these files — tests? radius? | <code>--situ</code>                    | ~410       | ~3K–132K   | 7.3×–324.2×  |
| Review this PR/diff                    | <code>--pr-context=REF</code>          | ~1.9K      | ~4.8K–51K  | 2.6×–27.5×   |

Figures are ~tokens (≈ bytes/4). Every row is scored same-correct-answer-or-it-doesn't-count — both sides were checked, not assumed — and every ratio is a **range**: the cheap end and the honest end of what an agent would actually read, never the single most flattering number.

// the cost lever

# Smaller answers that are also better answers

61.0%

fewer element bytes at top-50 — the --pack-signatures ladder,  
root-neutralised, re-derived on this repo every run

Ask for a symbol by NAME and the router uses the  
name-exact ranker:

**MRR** 0.797 → 0.990

**recall@1** 70.0% → 98.0%

name-shaped queries on src/, held-out  
labels, re-derived 2026-08-08

This figure survived its own audit: three corrections deep, re-derived  
root-neutrally after the original was shown to depend on how the  
corpus path was spelled — three spellings of one root read 18.6  
points apart before that subtraction. top-50 is the quotable number  
because the payload is top-50 whatever --top-k says.

And the pollution metric — fixture and generated paths  
contaminating results — is driven to 0.0% on the ranking lane  
while recall went UP, not down.

```
// the half no other tool ships
```

# The tripwire: honesty as an output contract

## `counts_floor="1"`

call edges come from source text — dynamic dispatch adds no edge, so every count that cannot claim completeness is labelled a floor, in the output itself

## `a zero is a measurement`

zero means none found, never none exists — and absent  $\neq$  0

## `truncation is disclosed`

every cap, page seam and budget cut is named in the header before you read a row

## `refusals teach`

a refused query names the flag, the problem, and a working example — never a bare error

```
$ ripwire . --callers=rankGraphTeleport

<callers of="rankGraphTeleport"
  defs="1" count="6"
  counts_floor="1">
<s t="fn" n="runEval" .../>
<s t="fn" n="rankGraph" .../>
...
</callers>
```

**The map grades itself before it answers.** This repository's own src/: files=109 symbols=3233 edges=10132 ambiguous=4768 unresolved=1097.

```
// the honesty marks stop being a promise and become a measurement
```

# Four levers accepted, one rejected on purpose

## macro edges

kParserVer 48

function-like #define invocations land role="macro" edges on disclosed-degraded symbols

## fn-pointer bindings

kParserVer 49

calls through unambiguous local function-pointer bindings now resolve

## using-declarations

kParserVer 51

re-exports emit role="import" references — r9 loss bucket 1

## shadow suppression

kParserVer 52

a local variable shadowing a function name stops stealing its use-sites — r9 loss bucket 2

## RTA-lite

reverted

instantiation-filtered CHA cone: refuted TWICE — the instantiated set crossed namespaces, and narrowed calls LOST their amb= mark

The two r9 levers, against the compiler oracle: **site precision 0.9046 → 0.9136, recall 0.9285 → 0.9412.**

**The fn-pointer lever raised unresolved= by 1,039. Disclosure, not regression:** call sites the resolver now RECOGNIZES but refuses to resolve are counted instead of silently ignored.

// how it stays true

# Proven, not promised

## 397 gate scripts

the suite runs on every push — plus determinism, cache-transparency and golden contracts; the gate count itself is gated against the runner's own loop

## byte-identical, always

two runs over the same tree produce the same bytes; warm equals cold. Enforced in CI, twice — Release AND a plain flavour, because NDEBUG once blinded a whole class of checks

## differential refactoring

a refactor must prove it changed nothing observable: two binaries, hundreds of argv vectors, stdout + stderr + exit codes byte-identical

## held-out labels, authored blind

eval labels were written by reading source before the ranker ever ran on them — so the eval is allowed to say the ranker is wrong. It has.

## sanitizer wall

ASan + UBSan + integer + float-cast, no-recover; TSan separately; leak suppression is one pinned file

## its own output is audited

the showcase capture is regenerated and adversarially re-read as a claims corpus — the methodology ships in docs/METHODOLOGY.md



// own the losses — they are in the README too

# Where it loses, published with the wins

**The public C++ number is lower than the private one it replaced**

SFML: strict file@10 28.7%, any@10 41.7%, first-hit MRR 0.21. Until 2026-08-07 this bullet compared against a ~89% any@10 private corpus that is no longer reproducible. That comparison is retired; the public number is the baseline going forward.

**--grep costs more than it saves**

+19.7% tokens / -11.2% — published next to the flag it indicts, and carrying the same private-corpus caveat as every figure in that source

**PageRank is the wrong co-change ranker**

3.8% recall@5 against 40.3% for plain lexical, and fusing the two made it worse — relatedness is lexical, importance is structural

**Strict multi-file localization stays hard**

single-file gold 73.4% vs multi-file 18.2% (held-out); the same cliff on every corpus — open headroom, said plainly

*A tool that publishes its counterexamples is a tool whose wins you can believe.*

```
// the single command for "is this code actually rotten?"
```

# Six families of evidence, counted — never blended

**structural**

shape: complexity, size, nesting, params, readability rank

--metrics

**lexical**

identifier text: the naming rules

--lint --naming-consistency

**confusion**

syntax: the atoms-of-confusion rules

--lint

**historical**

git change frequency, measured PER FILE

--hotspots

**colocation**

how much you must READ outside this file

--context-ratio

**state**

this function's OWN BODY touching non-local mutable state

--nonlocal-state

**Ranked by the COUNT of distinct families that fire** — never a weighted composite, because a composite lets one loud family impersonate six.

**4 of 6** is what the strict preset counts — the verb says so itself, families="6" enabled\_n="4". Two families did not clear the stability ladder.

--quality-panel[=strict|default|lenient] — and --quality-delta for the narrower question: what did MY change make worse?

// a new lens does not ship because it sounds plausible

# The calibration that disqualified a family

**+0.168**

the LARGEST cross-family correlation in the whole 6×6 matrix — widening the panel from four families to six did not raise it

**27,999**

eligible functions, five independent trees — two reported separately and never pooled, because one of them is this repository

**4 of 6**

families the strict preset actually counts — TWO were measured and held back from gating

**The uncomfortable result, kept.** The stability pass ran each family across a ladder of commits to ask whether it is steady enough to gate on. **colocation came out worse than historical on one ladder**, so neither gates: strict counts four of the six — a lens its own author wanted, measured, and demoted. The ladder was run precisely because that answer was possible; assuming it would pass is how a plausible axis becomes a permanent one.

**--field-affinity was NOT made a family either** — an exclusion by unit, not a failed measurement: it ranks struct FIELDS by co-access against declared layout, and the panel's unit is a function. Stated in the calibration rather than left for someone to notice.

```
// built for the agent's seat
```

# One command wires it into your agent

```
$ ripwire wrap claude
```

claude · cursor · codex · windsurf · gemini · aider — or --all to detect every one you have installed

## 30 MCP verbs

15 read verbs mirroring the CLI, 12 flagship reflexes (impact, uses, edit\_check, from\_trace, connect ...), 3 span-addressed edit verbs with a safety contract

## lazy-body handles

read verbs return signatures and a stable handle; the agent fetches a body only when it decides it needs one — names by default, bytes on request

## 18 agent skills

moment-matched workflows (orient, navigate, change-check, quality-bar ...) — wrap prints the recipe, skills/install.sh installs them

## 11 orchestrator loops

copy-paste prompts in prompts/: run the same audit, eval and head-to-head machinery that built this tool, on your own repository

**wrap does not just print JSON:** it probes what that agent already has, emits the config in that agent's own shape, and includes a use-when blurb so the agent knows which verb fires at which moment — not just that a server exists.

the MCP server exposes the same deterministic engine — one index, shared with the CLI, staleness-checked

// standing on giants

# The research inside — classic and current

**34 repositories + 54 papers folded** · and a labelled survey of 221 tools that contributed nothing, which says so — every row with the lesson taken and where it lives: docs/LINEAGE.md

**1976**     **Cyclomatic complexity — McCabe**  
the metric behind --hotspots

**1994**     **Okapi BM25 — Robertson · Spärck Jones**  
the lexical ranker (twice: subtoken+body, whole-name)

**1998**     **Personalized PageRank — Page · Brin**  
the headline importance signal

**2008**     **Louvain modularity — Blondel et al.**  
the module / community map

**2009**     **Reciprocal Rank Fusion — Cormack et al.**  
deterministic signal fusion (--rank-by=rfr)

**2011**     **Readability model — Posnett · Hindle · Devanbu**  
the lexical family of the quality panel

**2019**     **Delta Maintainability Model — di Biase et al.**  
--dmm: one comparable number per change

**TDAD** · **arXiv 2603.17973**

a static test-map cut agent-caused regressions 6.08% → 1.82%; prose TDD instructions alone made agents WORSE → the shape of --test-gate

**What-to-Retrieve** · **arXiv 2503.20589**

retrieval selection beats retrieval volume for coding agents → the selection-over-dumping thesis

**LocAgent / Loc-Bench (2025)**

the localization metric (strict Acc@k) and the frozen 560-instance dataset every accuracy number here is scored on

**scip-clang** · **Serena / clangd**

the compiler-grade oracle round 9 graded both tools against — an outside referee, not a rival

**tree-sitter**

the incremental GLR parsing substrate — 16 grammars vendored, one parser per thread

counts gated in-repo (readmedriftcheck arm E derives them from LINEAGE.md's own tables) — the lineage is the design rationale, not decoration

// do not take any of it on trust

# Every claim, and the command that re-derives it

156 long flags · 23 slides

```
bash test/deckclaimcheck.sh
```

every --flag named here exists

```
bash test/deckcheck.sh
```

67.0% fewer element bytes

```
bash test/showcasecapturecheck.sh
```

397 gate scripts

```
bash test/manifestcheck.sh
```

34 repos · 54 papers · 221 surveyed

```
bash test/readmedriftcheck.sh
```

the ten moments, any row

```
ripwire . --callers=SYM | wc -c
```

the head-to-head table

```
bench/headtohead/r4-2026-08-06/
```

the oracle round

```
bench/headtohead/r9-2026-08-09/RESULTS.md
```

**The deck is generated, and the generator is gated.** `present/deck5_ripwire_build.js` is scanned for fabricated flags and its slide count is derived from its own `addSlide()` calls — a deck that drifts from the binary fails the suite.

`docs/EVALS.md` is the source of truth; `$7` is the counterexamples, `$8` the claims this project refuses to make

```
// sixty seconds to the first ranked map
```

# Quickstart

```
git clone <repo> && cd ripwire  
cmake -S . -B build && cmake --build build -j          # hermetic: works with the network off  
./build/ripwire --help
```

```
ripwire .
```

the ranked map — start here on any unfamiliar repo

```
ripwire . --for="cache invalidation"
```

the task lens: what to touch, ranked

```
ripwire . --callers=someFunction
```

who calls it — with the honesty marker attached

```
ripwire . --quality-panel
```

is this code actually rotten? six families, one shortlist

```
ripwire . --test-gate
```

before you commit: exactly which tests must run

```
ripwire wrap claude
```

wire it into your agent, in that agent's own config shape

**ripgrep** for the speed. **tripwire** for the honesty. Apache-2.0.